



in collaboration with



LAB 03: Investigating Process Lifecycles and OS Interaction

Aaisma Wagle

Bsc (Hons) Computer Science with Artificial Intelligence

Softwarica College of IT and E-commerce

ST5003CMD Operating Systems, Security, and Networks

Rupak Rajbanshi

26th September, 2026

TABLE OF CONTENT

INTRODUCTION	1
THE LONG-RUNNING PROCESS	2
A. C Source Code	2
B. Terminal Execution & Verification	3
PROCESS IDENTITY (PID AND PPID)	4
A. C Source Code	4
B. Terminal Execution & Verification	5
EXIT CODES AND OS FEEDBACK	6
A. C Source Code	6
B. Terminal Execution & Verification	6
STANDARD I/O STREAMS	7
A. C Source Code	7
B. Terminal Execution & Verification	7
CONDITIONAL EXECUTION AND TERMINATION	8
A. C Source Code	8
B. Terminal Execution & Verification	9
CONCLUSION	9
GITHUB REPOSITORY	9

LIST OF FIGURES

Figure 01: Make LAB 03 Folder to Work With	1
Figure 02: apt update and gcc Version Check	1
Figure 03: Make task1_alive.c File	2
Figure 04: task1_alive.c File in Nano Editor	2
Figure 05: task1_alive.c File View using cat Command	2
Figure 06: Process Creation and Monitoring of task_alive using ps	3
Figure 07: New Folder task2 Made	4
Figure 08: task2_identity.c in Nano Editor	4
Figure 09: task2_identity.c View using Cat Command	4
Figure 10: Process Identification and Verification using ps	5
Figure 11: Make task3_exit.c File in Folder ‘task3’	6
Figure 12: task3_exit.c File in Nano Editor	6
Figure 13: task3_exit.c View using Cat Command	6
Figure 14: Compilation, Execution, and Exit Status Verification of task3_exit.c	6
Figure 15: Make task4_input.c File in Folder ‘task3’	7
Figure 16: task4_input.c File in Nano Editor	7
Figure 17: task4_input.c View using Cat Command	7
Figure 18: Terminal Execution of task4_input.c	7
Figure 19: task5_control.c File in Nano Editor	8
Figure 20: task5_control.c View using Cat Command	8
Figure 21: Terminal Execution and Exit Status of task5_control.c	9

INTRODUCTION

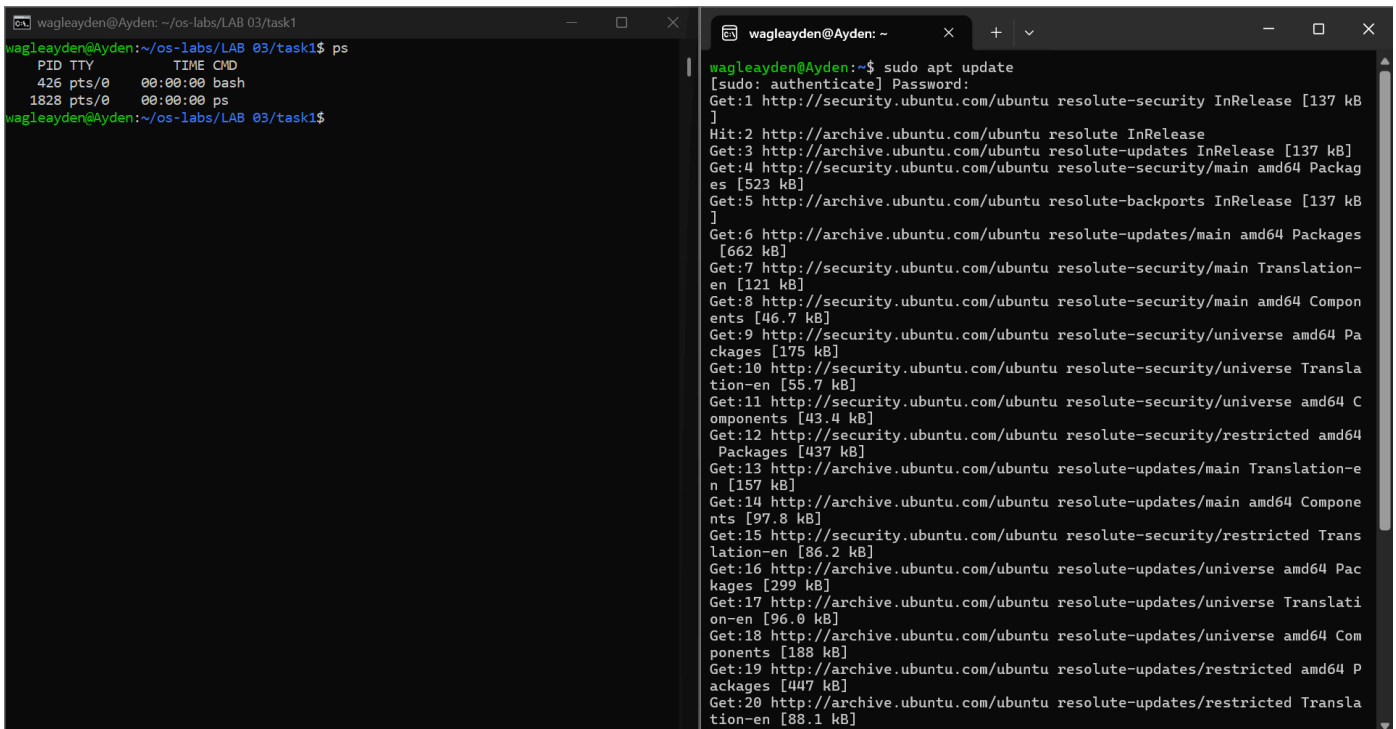
This is the lab report of Operating Systems, Security and Networks which explores how a running C program interacts with the Linux operating system. While earlier labs focused on writing and compiling C code, this lab looks at what happens after compilation. It shows how the OS creates a process, assigns it an identity, tracks it while it runs, and receives feedback from it when it exits. Five short programs were used to demonstrate this. For this, we made a repository called 'LAB 03'.

Sudo mkdir <repository name> //For creating a new folder. Sudo is used to access files that regular user account doesn't have access to

ls //list all files in the directory

```
wagleayden@Ayden:~/os-labs$ ls
C Compile  'LAB 02'  week1
wagleayden@Ayden:~/os-labs$ sudo mkdir 'LAB 03'
wagleayden@Ayden:~/os-labs$ ls
C Compile  'LAB 02'  'LAB 03'  week1
wagleayden@Ayden:~/os-labs$
```

Figure 01: Make LAB 03 Folder to Work With



The image shows two terminal windows. The left window, titled 'wagleayden@Ayden: ~/os-labs/LAB 03/task1', displays the output of the 'ps' command, showing three processes: 'bash' (PID 426) and 'ps' (PID 1828). The right window, titled 'wagleayden@Ayden: ~', shows the output of 'sudo apt update', which lists various Ubuntu repositories and the size of the data being downloaded, such as 'resolute-security InRelease' and 'resolute-updates InRelease'.

Figure 02: apt update and gcc Version Check

THE LONG-RUNNING PROCESS

A. C Source Code

Here, a new file called ‘*task1_alive.c*’ was made. It is compiled into the executable *task_alive*. Running the program prints “I am starting....” and waits 30 to print “I am finished....”. The **ps** and **ps aux** commands are used to monitor the process table, and **ps aux | grep task_alive** confirms that the program is running as a process.

The `sleep()` function is defined in `<unistd.h>` and `<stdio.h>` defines `scanf()` and `printf()`.

Sudo touch <filename> //For creating a new file.

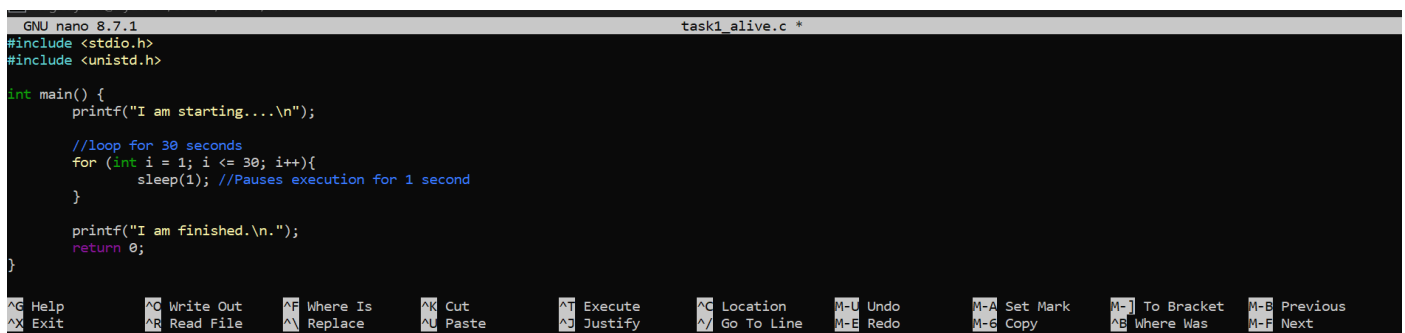
Sudo nano <filename> //To open the file in nano editor to write or edit

It is saved using **Ctrl + X, Y** and **Enter**

cat <filename> //Prints a file's text directly to the terminal screen.

```
wagleayden@Ayden:~/os-labs/LAB 03$ ls
wagleayden@Ayden:~/os-labs/LAB 03$ sudo mkdir task1
wagleayden@Ayden:~/os-labs/LAB 03$ cd task1
wagleayden@Ayden:~/os-labs/LAB 03/task1$ sudo touch task1_alive.c
wagleayden@Ayden:~/os-labs/LAB 03/task1$ ls
task1_alive.c
wagleayden@Ayden:~/os-labs/LAB 03/task1$
```

Figure 03: Make *task1_alive.c* File



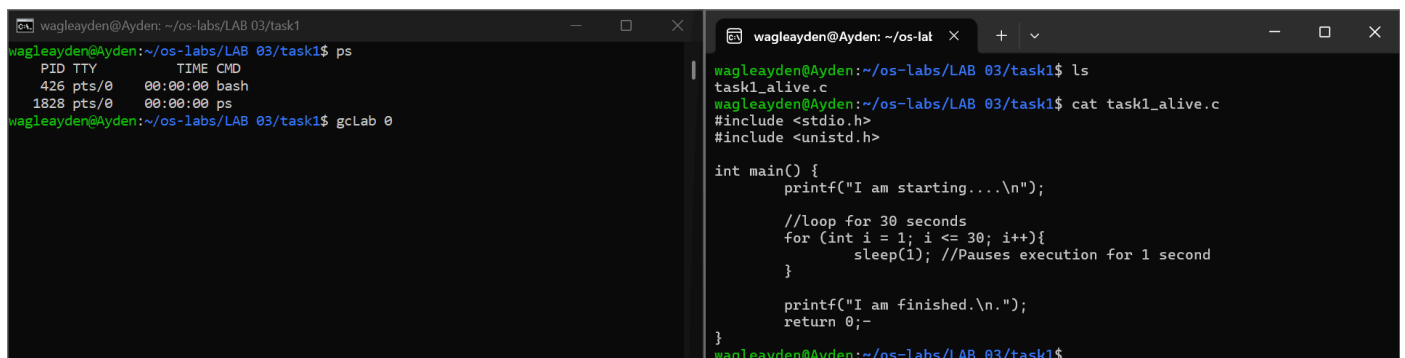
```
GNU nano 8.7.1 task1_alive.c *
#include <stdio.h>
#include <unistd.h>

int main(){
    printf("I am starting....\n");

    //loop for 30 seconds
    for (int i = 1; i <= 30; i++){
        sleep(1); //Pauses execution for 1 second
    }

    printf("I am finished.\n.");
    return 0;
}
```

Figure 04: *task1_alive.c* File in Nano Editor



```
wagleayden@Ayden: ~/os-labs/LAB 03/task1
wagleayden@Ayden:~/os-labs/LAB 03/task1$ ps
  PID TTY          TIME CMD
   426 pts/0    00:00:00 bash
  1828 pts/0    00:00:00 ps
wagleayden@Ayden:~/os-labs/LAB 03/task1$ gcLab 0

wagleayden@Ayden:~/os-labs/LAB 03/task1$ ls
task1_alive.c
wagleayden@Ayden:~/os-labs/LAB 03/task1$ cat task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting....\n");

    //loop for 30 seconds
    for (int i = 1; i <= 30; i++){
        sleep(1); //Pauses execution for 1 second
    }

    printf("I am finished.\n.");
    return 0;
}
wagleayden@Ayden:~/os-labs/LAB 03/task1$
```

Figure 05: *task1_alive.c* File View using *cat* Command

B. Terminal Execution & Verification

Terminal 01:

ps //Shows processes for the current shell.

ps aux //Shows all processes with details like user, PID, CPU, memory, etc.

ps aux | grep <executable file>

//Filters the list to only show your program

Terminal 02:

sudo gcc hello.c -o <executable binary file>

//Compiles the source code into a runnable program

./<executable binary file>

//Runs the binary file in the terminal

```
wagleyayden@Ayden: ~/os-labs/LAB 03/task1$ ps
  PID TTY          TIME CMD
  426 pts/0    00:00:01 bash
 2369 pts/0    00:00:00 ps
wagleyayden@Ayden:~/os-labs/LAB 03/task1$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root             1   0.1  0.4  24388 15464 ?        Ss   05:26   0:03 /sbin/init
root             2   0.0  0.0   3180  2208 hvc0     S1+  05:26   0:00 /init
root             6   0.0  0.0   3180  2040 hvc0     S1+  05:26   0:00 plan9 --control-s
root            49   0.0  0.4  50416 17004 ?        S<s  05:26   0:01 /usr/lib/systemd/
systemd+       89   0.0  0.3  22416 14544 ?        Ss   05:26   0:00 /usr/lib/systemd/
root            97   0.0  0.3  35324 12364 ?        Ss   05:26   0:02 /usr/lib/systemd/
root           150   0.0  0.0   2888  2076 ?        Ss   05:26   0:00 /bin/sh /usr/lib/
root           151   0.0  0.0   4472  3212 ?        Ss   05:26   0:00 /usr/sbin/cron -f
message+      152   0.0  0.1   8800  5568 ?        Ss   05:26   0:00 @dbus-daemon --sy
root           161   0.0  0.7  44096 29864 ?        Ss   05:26   0:00 /usr/bin/python3
root           183   0.0  0.2  18560  9524 ?        Ss   05:26   0:00 /usr/lib/systemd/
syslog        209   0.0  0.1  220548  5920 ?        Ssl  05:26   0:00 /usr/sbin/rsyslog
root          262   0.0  0.0   5492  2848 tty1     Ss+  05:26   0:00 /usr/sbin/agetty
root          277   0.0  0.8 123460 32792 ?        Ssl  05:26   0:00 /usr/bin/python3
_chrony       339   0.1  0.1  20424  7080 ?        S    05:26   0:03 /usr/sbin/chronyd
_chrony       343   0.0  0.0  12096  2492 ?        S    05:26   0:00 /usr/sbin/chronyd
root          423   0.0  0.0   3184  1112 ?        Ss   05:26   0:00 /init
root          424   0.0  0.0   3200  1252 ?        S    05:26   0:00 /init
wagleyay+    426   0.0  0.1   6268  5680 pts/0    Ss   05:26   0:01 -bash
root         427   0.0  0.1   8648  5676 ?        Ss   05:26   0:00 login -- wagleyayd
wagleyay+    472   0.0  0.3  22340 13336 ?        Ss   05:26   0:00 /usr/lib/systemd/
wagleyay+    474   0.0  0.1  22916  4124 ?        S    05:26   0:00 (sd-pam)
wagleyay+    500   0.0  0.1   6136  5596 pts/1    Ss+  05:26   0:00 -bash
root        1833   0.0  0.0   3184  1120 ?        Ss   06:13   0:00 /init
root        1834   0.0  0.0   3200  1260 ?        S    06:13   0:00 /init
wagleyay+   1839   0.0  0.1   6268  5688 pts/2    Ss   06:13   0:00 -bash
polkitd      1932   0.0  0.2 306444  8432 ?        Ssl  06:14   0:00 /usr/lib/polkit-1
wagleyay+   2406   0.0  0.0   2768  1820 pts/2    S+   06:26   0:00 ./task_alive
wagleyay+   2412   0.0  0.1   7160  4432 pts/0    R+   06:26   0:00 ps aux
wagleyayden@Ayden:~/os-labs/LAB 03/task1$ ps aux | grep task_alive
wagleyay+   2406   0.0  0.0   2768  1820 pts/2    S+   06:26   0:00 ./task_alive
wagleyay+   2423   0.0  0.0   4260  2408 pts/0    S+   06:27   0:00 grep --color=auto
task_alive
wagleyayden@Ayden:~/os-labs/LAB 03/task1$
```

```
wagleyayden@Ayden: ~/os-lab x + v
wagleyayden@Ayden:~/os-labs/LAB 03/task1$ sudo gcc task1_alive.c -o task_alive
wagleyayden@Ayden:~/os-labs/LAB 03/task1$ ./task_alive
I am starting....
```

Figure 06: Process Creation and Monitoring of `task_alive` using `ps`

This figure shows the compilation and execution of the program `task1_alive`. It prints “I am starting....” when it is run. The `ps`, `ps aux`, and `ps aux | grep task_alive` commands are used to monitor the process table and confirm that the `task_alive` executable is running as a process in the system.

PROCESS IDENTITY (PID AND PPID)

A. C Source Code

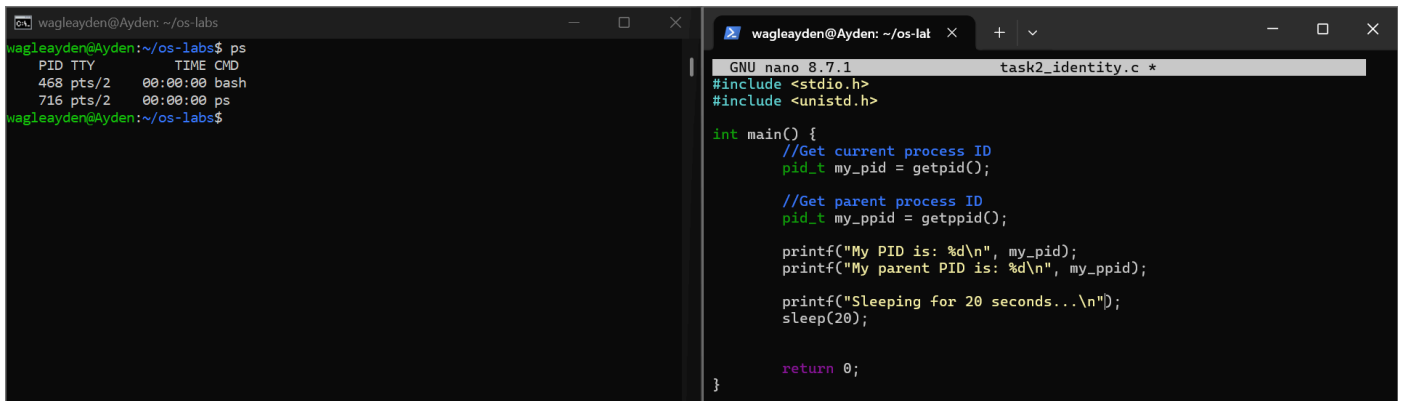
Before starting on the second task, a new folder called 'task2' was made.

```
cd .. //Goes one step back to the repository the current repository it was in
cd <repository> //enters the folder
```

```
wagleayden@Ayden:~/os-labs/LAB 03/task1$ ls
task1_alive.c  task_alive
wagleayden@Ayden:~/os-labs/LAB 03/task1$ cd ..
wagleayden@Ayden:~/os-labs/LAB 03$ sudo mkdir task2
[sudo: authenticate] Password:
wagleayden@Ayden:~/os-labs/LAB 03$ ls
task1  task2
wagleayden@Ayden:~/os-labs/LAB 03$ |
```

Figure 07: New Folder task2 made

Here, a new file called 'task2_identity.c' was made. The source file *task2_identity.c* is compiled into the executable *task2_alive*. Running the program prints the pid and ppid and waits 20 seconds to exit. The **ps** and **ps aux** commands are used to monitor the process table, and **ps aux | grep task_alive** confirms that the program is running as a process.



```
GNU nano 8.7.1 task2_identity.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    //Get current process ID
    pid_t my_pid = getpid();

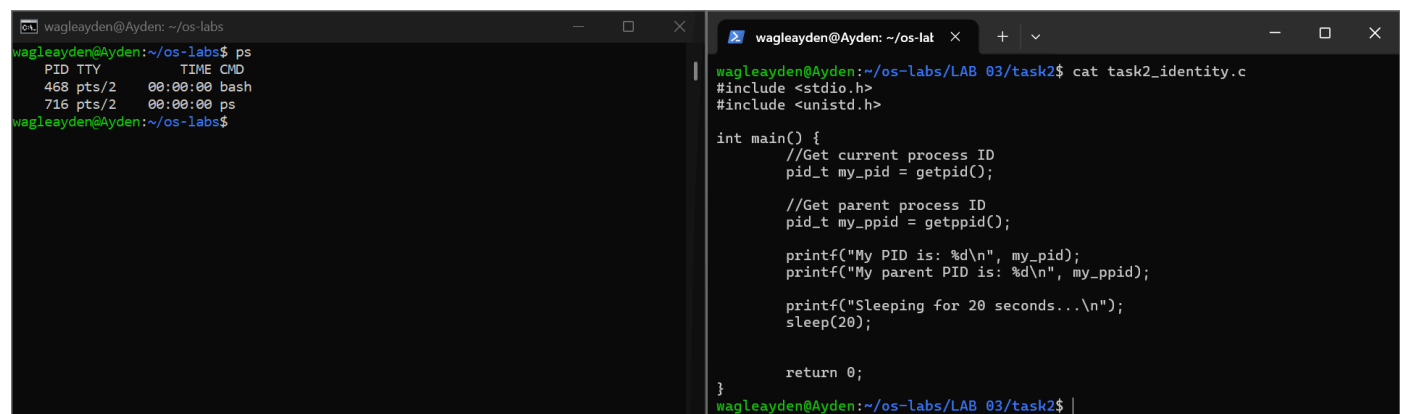
    //Get parent process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 08: task2_identity.c in Nano Editor



```
wagleayden@Ayden:~/os-labs/LAB 03/task2$ cat task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    //Get current process ID
    pid_t my_pid = getpid();

    //Get parent process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 09: task2_identity.c View using Cat Command

B. Terminal Execution & Verification

Terminal 01:

```
ps aux | grep <executable file>
```

```
//Filters the list to only show your program
```

```
ps -p <PID> -o pid,ppid,cmd
```

```
//See PID, PPID and CMD by its PID
```

Terminal 02:

```
sudo gcc hello.c -o <executable binary file>
```

```
//Compiles the source code into a runnable program
```

```
./<executable binary file>
```

```
//Runs the binary file in the terminal
```

The figure shows two terminal windows. The left window displays the output of the command `ps aux`, which lists all running processes with columns for USER, PID, %CPU, %MEM, VSZ, RSS, TTY, STAT, START, TIME, and COMMAND. The output shows various system processes like `/sbin/init`, `/usr/lib/systemd/systemd`, and `/usr/sbin/cron`. Below this, the command `ps -p 1474 -o pid,ppid,cmd` is executed, showing the details of the process with PID 1474, which is `./task2_identity` with a parent PID of 323. The right window shows the execution of `sudo gcc task2_identity.c -o task2_identity` and then `./task2_identity`. The output of the program shows "My PID is: 1474" and "My parent PID is: 323", confirming the relationship between the running process and its parent shell.

Figure 10: Process Identification and Verification using *ps*

This figure shows the execution of the program `task2_identity`, which prints its own PID and PPID. The `ps -p <PID> -o pid,ppid,cmd` command is then used to verify the process ID and its parent process ID in the Linux environment, confirming the relationship between the running process and its parent shell.

EXIT CODES AND OS FEEDBACK

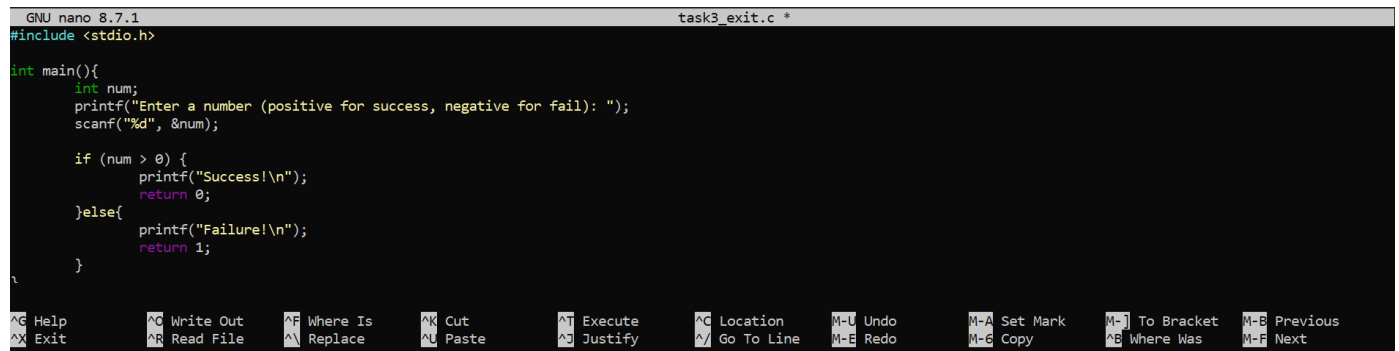
A. C Source Code

A new repository called ‘task3’ and file ‘task3_exit.c’ is made.

```
wagleayden@Ayden:~/os-labs/LAB 03$ sudo mkdir task3
wagleayden@Ayden:~/os-labs/LAB 03$ cd task3
wagleayden@Ayden:~/os-labs/LAB 03/task3$ sudo touch task3_exit.c
wagleayden@Ayden:~/os-labs/LAB 03/task3$ sudo nano task3_exit.c
wagleayden@Ayden:~/os-labs/LAB 03/task3$
```

Figure 11: Make task3_exit.c File in Folder ‘task3’

The program task3_exit.c is created in the Nano editor. It prompts the user to enter a number, prints “Success” if the number is positive and “Failure” if negative. Then it returns exit codes 0 or 1 according to the operating system.



```
GNU nano 8.7.1 task3_exit.c *
#include <stdio.h>

int main(){
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success!\n");
        return 0;
    }else{
        printf("Failure!\n");
        return 1;
    }
}
```

Figure 12: task3_exit.c File in Nano Editor

```
wagleayden@Ayden:~/os-labs/LAB 03/task3$ sudo nano task3_exit.c
wagleayden@Ayden:~/os-labs/LAB 03/task3$ cat task3_exit.c
#include <stdio.h>

int main(){
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success!\n");
        return 0;
    }else{
        printf("Failure!\n");
        return 1;
    }
}
```

Figure 13: task3_exit.c View using Cat Command

B. Terminal Execution & Verification

The program is compiled using **gcc** and executed in the terminal. The user enters both positive and negative values to test the program. The **echo \$?** command is used to verify the exit status returned to the operating system: 0 for success and 1 for failure. It demonstrates correct program behavior.

```
wagleayden@Ayden:~/os-labs/LAB 03/task3$ sudo gcc task3_exit.c -o task3_exit
wagleayden@Ayden:~/os-labs/LAB 03/task3$ ./task3_exit
Enter a number (positive for success, negative for fail): 3
Success!
wagleayden@Ayden:~/os-labs/LAB 03/task3$ echo $?
0
wagleayden@Ayden:~/os-labs/LAB 03/task3$ ./task3_exit
Enter a number (positive for success, negative for fail): -3
Failure!
wagleayden@Ayden:~/os-labs/LAB 03/task3$ echo $?
1
```

Figure 14: Compilation, Execution, and Exit Status Verification of task3_exit.c

STANDARD I/O STREAMS

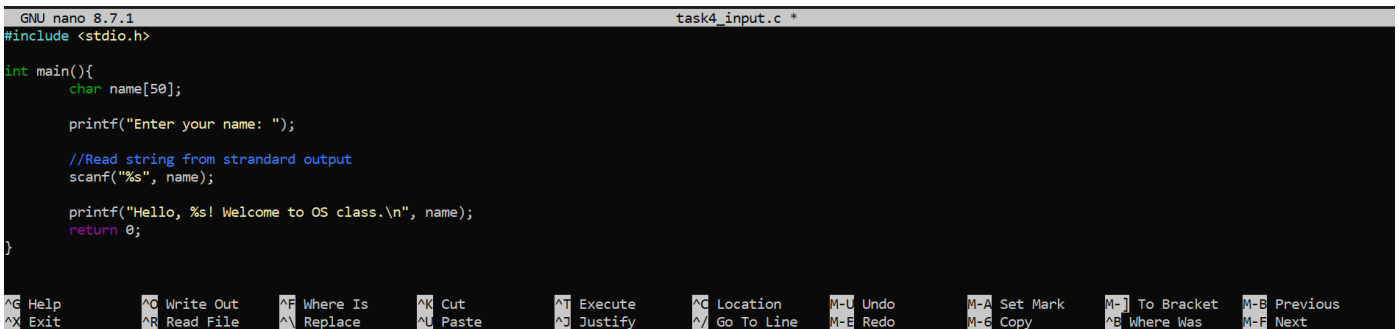
A. C Source Code

A new repository called 'task4' and file 'task4_input.c' is made.

```
wagleyden@Ayden:~/os-labs/LAB 03$ ls
task1 task2 task3
wagleyden@Ayden:~/os-labs/LAB 03$ sudo mkdir task4
wagleyden@Ayden:~/os-labs/LAB 03$ cd task4
wagleyden@Ayden:~/os-labs/LAB 03/task4$ sudo touch task4_input.c
```

Figure 15: Make task4_input.c File in Folder 'task4'

The program prompts the user to enter their name and then displays a personalized greeting. It uses scanf() and printf() functions for input and output. After saving the file, the code was viewed using the cat command to confirm its contents and compiled with the GCC compiler.



```
GNU nano 8.7.1 task4_input.c *
#include <stdio.h>

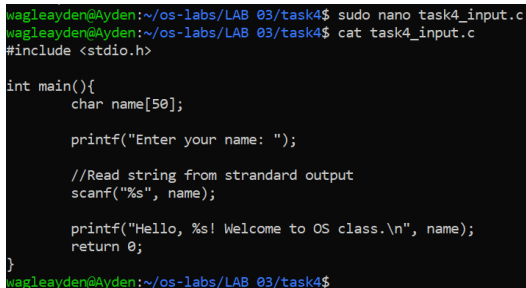
int main(){
    char name[50];

    printf("Enter your name: ");

    //Read string from standard output
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS class.\n", name);
    return 0;
}
```

Figure 16: task4_input.c File in Nano Editor



```
wagleyden@Ayden:~/os-labs/LAB 03/task4$ sudo nano task4_input.c
wagleyden@Ayden:~/os-labs/LAB 03/task4$ cat task4_input.c
#include <stdio.h>

int main(){
    char name[50];

    printf("Enter your name: ");

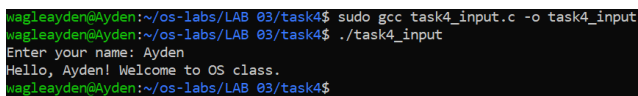
    //Read string from standard output
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS class.\n", name);
    return 0;
}
wagleyden@Ayden:~/os-labs/LAB 03/task4$
```

Figure 17: task4_input.c View using Cat Command

B. Terminal Execution & Verification

The program is compiled using gcc and executed in the terminal. The user enters a name and the output is a personalized message welcoming them to OS class. It shows the proper use of standard input and output streams in C.



```
wagleyden@Ayden:~/os-labs/LAB 03/task4$ sudo gcc task4_input.c -o task4_input
wagleyden@Ayden:~/os-labs/LAB 03/task4$ ./task4_input
Enter your name: Ayden
Hello, Ayden! Welcome to OS class.
wagleyden@Ayden:~/os-labs/LAB 03/task4$
```

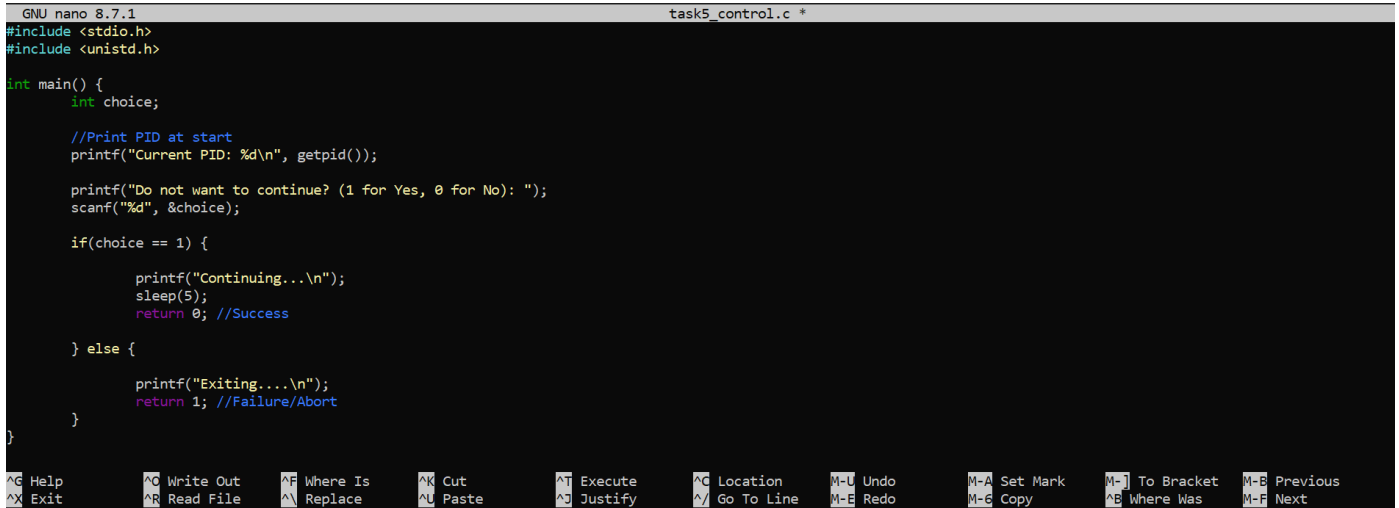
Figure 18: Terminal Execution of task4_input.c

CONDITIONAL EXECUTION AND TERMINATION

A. C Source Code

A new repository called 'task5' and file 'task5_control.c' is made.

The program prompts the user to enter their name and then displays a personalized greeting. It uses scanf() and printf() functions for input and output. After saving the file, the code was viewed using the cat command to confirm its contents and compiled with the GCC compiler.



```
GNU nano 8.7.1 task5_control.c *
#include <stdio.h>
#include <unistd.h>

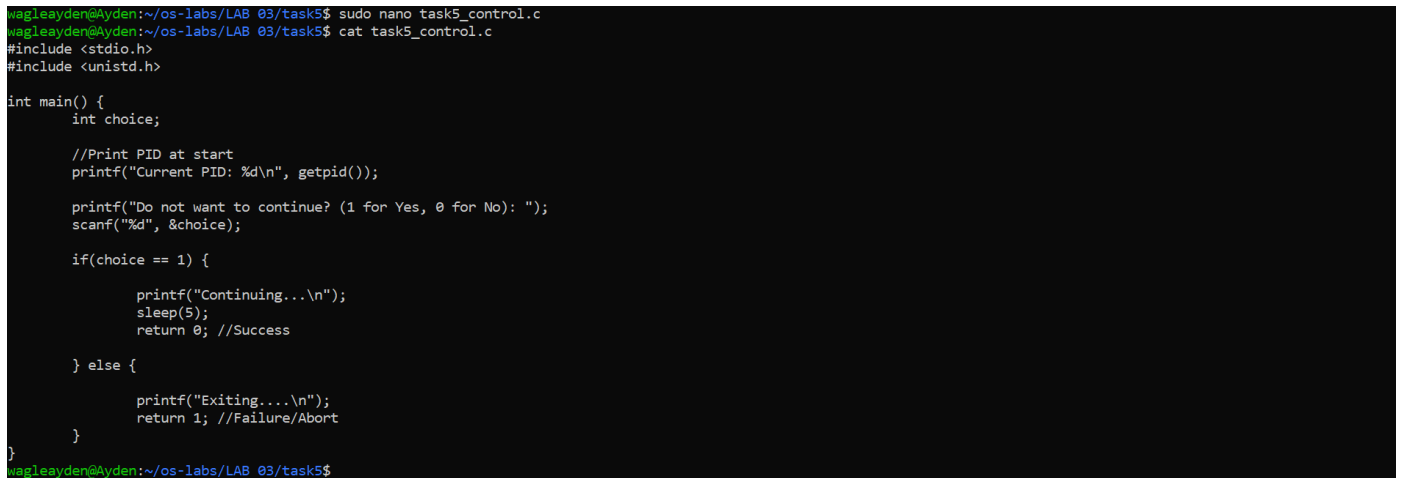
int main() {
    int choice;

    //Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do not want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if(choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; //Success
    } else {
        printf("Exiting...\n");
        return 1; //Failure/Abort
    }
}
```

Figure 19: task5_control.c File in Nano Editor



```
wagleayden@Ayden:~/os-labs/LAB 03/task5$ sudo nano task5_control.c
wagleayden@Ayden:~/os-labs/LAB 03/task5$ cat task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    //Print PID at start
    printf("Current PID: %d\n", getpid());

    printf("Do not want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if(choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; //Success
    } else {
        printf("Exiting...\n");
        return 1; //Failure/Abort
    }
}
wagleayden@Ayden:~/os-labs/LAB 03/task5$
```

Figure 20: task5_control.c View using Cat Command

B. Terminal Execution & Verification

The program is compiled using **gcc** and executed in the terminal. The user enters a name and the output is a personalized message welcoming them to OS class. It shows the proper use of standard input and output streams in C.

```
wagleyden@Ayden:~/os-labs/LAB 03/task5$ sudo gcc task5_control.c -o task5_control
wagleyden@Ayden:~/os-labs/LAB 03/task5$ ./task5_control
Current PID: 3672
Do not want to continue? (1 for Yes, 0 for No): 1
Continuing...
wagleyden@Ayden:~/os-labs/LAB 03/task5$ echo $?
0
wagleyden@Ayden:~/os-labs/LAB 03/task5$ ./task5_control
Current PID: 3691
Do not want to continue? (1 for Yes, 0 for No): 0
Exiting....
wagleyden@Ayden:~/os-labs/LAB 03/task5$ echo $?
1
wagleyden@Ayden:~/os-labs/LAB 03/task5$
```

Figure 21: Terminal Execution and Exit Status of task5_control.c

CONCLUSION

This lab gave practical experience with how the operating system manages a C program's lifecycle, from creation to termination. Monitoring *task1_alive* with **ps** showed a compiled program running as a live process. *task2_identity* confirmed the PID/PPID relationship between a process and its parent shell. *task3_exit* and *task5_control* showed how a program's return value is passed to the OS as an exit status. **echo \$?** confirms 0 for success and non-zero values like 1 for failure. *task4_input* demonstrated the use of standard input and output streams via `scanf()` and `printf()`.

Overall, the lab met its objective of understanding process behaviour and exit codes, which are essential background for the systems programming and process management topics covered later in this module.

GITHUB REPOSITORY

[GitHub Repository](#)